

Reviewing What It Gave You

Every module so far assumed you could tell when the output was wrong.

What the research actually found

- ◆ **A randomised trial.** Experienced developers were 19% slower using AI tools.
- ◆ **They believed the opposite.** Predicted 24% faster. Reported about 20% faster afterwards.
- ◆ **Defect rates.** AI-coauthored changes carry noticeably more issues than others.
- ◆ **And yet.** Most developers do not merge without reviewing. The instinct is right.

THE POINT

The gap between being slower and feeling faster is the finding. It applies to correctness too.

Fluent output reads as **correct** output. It always has.

Why 'review this code' does nothing

- ◆ **It has no target.** A general request gets a general answer.
- ◆ **It defaults to style.** Naming, formatting, comments. All true, none important.
- ◆ **It has already agreed with itself.** It wrote this. It is not a fresh reader.
- ◆ **Name the defect class** and the hit rate changes completely.

THE POINT

Asking it to check its own work is asking the same reasoning to reach a different conclusion.

Name the defect. Do not ask for a review.

What you actually type

One class per pass. Run it several times.

Check the code below for ONE class of defect only.
Ignore style, naming, formatting and performance.

This pass: operator precedence in WHERE clauses.

For every WHERE clause mixing AND and OR, show me how it actually parses, and whether that matches the apparent intent based on the indentation.

Quote each clause. State the parse. State the likely intent.
If they match, say 'matches' and move on.

Do not report anything outside this defect class.

HOW TO USE IT

Run this five times with five different classes. It beats one general review every time.

Defect classes worth naming

One pass each. Pick the ones your codebase actually suffers from

- ◆ **Operator precedence** in clauses mixing AND and OR.
- ◆ **Null propagation** through concatenation, arithmetic and comparison.
- ◆ **Join cardinality** against the declared grain.
- ◆ **Literal consistency** for the same code list across procedures.
- ◆ **Dialect drift** where a function is not native to the target engine.

THE RESULT

Every one of these appeared somewhere in this course. None would be found by a general review.

Null propagation, found by naming it

A general review pass read straight past this.

```
01 -- The dimension builds the key like this:
02 f_hash(src_key || UPPER(part) || NVL(UPPER(val_type),''))
03
04 -- The fact builds the same key like this:
05 f_hash(src_key || UPPER(part) || NVL(UPPER(val_type)),'')
06
07 -- Look at the closing bracket. The empty string is outside
08 -- the NVL, so NVL gets one argument and never defaults.
09 -- Null valuation type gives a null key. The join misses.
```

WHY TARGETING WORKS

Two characters apart, in two files, three hundred lines from each other.

Stop asking whether it is right. Ask it to show you it is **wrong**.

Same code, two questions

WHAT YOU NORMALLY ASK

Is this correct?

- Invites agreement
- Finds reasons it works
- Caveats arrive as reassurance
- Confirms what you already hoped

WHAT TO ASK INSTEAD

Show me it breaks

- Invites a counter-example
- Finds the input that fails
- Caveats arrive as conditions
- Gives you something to test

THE TIEBREAKER

The second question returns data conditions. Data conditions are checkable. Opinions are not.

What you actually type

Assume it is broken. Make it find how.

Assume the query below is wrong. Your job is to find the input that breaks it. Do not defend it.

Give me five data conditions that would make this return an incorrect result, ordered by how likely they are in a real warehouse. For each:

- The condition, stated concretely
- What the query would return
- What it should return
- A query that tells me whether the condition exists

If you cannot find five, say so. Do not pad the list.

THE BULLET THAT MATTERS

The fourth bullet turns the whole exercise from speculation into a morning's checking.

Last one. Make it tell you what it decided
for you.

What you actually type

Run this at the end of every session that produced code.

```
You have just written this query. Before I use it, list every
decision you made that I did not specify.
```

- Anything ambiguous in the spec that you resolved
- Any behaviour you chose where more than one was valid
- Anything you assumed about the data

```
For each, state what you chose and what the alternative was.
```

```
Then: which of these decisions would change the output if I
picked the other option? Order by size of effect.
```

```
If you made no such decisions, say so plainly.
```

WHAT TO DO WITH IT

This is the comment block that goes at the top of the query. It costs twenty seconds.

What review cannot catch, at all

- ◆ **A rule that is missing entirely.** Nothing in the code points at what is absent.
- ◆ **A code list that is stale.** The syntax is fine. The world moved.
- ◆ **The wrong business definition.** Correctly implemented, and not what was wanted.
- ◆ **All three need a person,** or a query against real data. Never another review pass.

THE BOUNDARY

If a second review pass would have found it, the first pass was badly targeted. If not, stop reviewing.

You ask for a general review of a 200 line procedure. You get eleven observations. Nine are about naming and formatting. Two suggest adding error handling.

The procedure contains a WHERE clause with a precedence bug you already know about.

What does this tell you?

- ☐ A The model is not capable of finding precedence bugs
- ☐ B The prompt had no target, so it defaulted to surface issues
- ☐ C The procedure is too long for one review pass
- ☐ D Precedence bugs are genuinely hard to detect

◆ PAUSE AND DECIDE

What does this tell you?

- A

 Not capable
Test it. Ask the targeted precedence prompt on the same code and watch it find the bug.
- B

 No target, surface answer
A general request gets a general answer, and style is the easiest general answer available.
- C

 Too long
Length matters and it is not the issue here. A targeted pass finds it at the same length.
- D

 Genuinely hard
It is hard for a human reading quickly. It is easy when you name it as the thing to look for.

THE SIGNAL

The same model, the same code, one changed instruction. That is the finding worth keeping.

The adversarial read returns five data conditions that would break the query. You run the check queries. Four conditions do not exist in your data. The fifth does.

The fifth is ranked as the least likely of the five.

What do you conclude about the ranking?

- ☐ A The ranking was wrong and should be ignored in future
- ☐ B The ranking was a prior, and your data is the evidence
- ☐ C The check query for the fifth condition is probably faulty
- ☐ D Four out of five is a good hit rate

◆ PAUSE AND DECIDE

What do you conclude about the ranking?

- A

 Ignore rankings
Overcorrects. The ranking told you what order to check in, which is all it was ever for.
- B

 A prior, updated by data
It ranked by likelihood in a typical warehouse. Yours is not typical, and now you know how.
- C

 Faulty check query
Worth a glance and it is not the interesting reading. Doubting the result you did not expect is a habit.
- D

 Good hit rate
Misreads the exercise. Ruling four out is the value, not a score to be graded on.

THE SIGNAL

The ranking sets your checking order. Your data decides. That is the division of labour.

The assumption disclosure returns four decisions. Three are trivial. The fourth says it assumed a code list was current, because the spec did not date it.

You recognise this. It is the same code list from the reconciliation module.

What is the right response?

- ☐ A Nothing new. You already knew about that list
- ☐ B Put all four in a comment at the top of the query
- ☐ C Go back and get the list verified before shipping
- ☐ D Both B and C

◆ PAUSE AND DECIDE

What is the right response?

- A

 Nothing new
True for you and false for everyone who reads this query after you leave.
- B

 Comment all four
Cheap and permanent. Even the trivial three cost nothing to carry.
- C

 Verify before shipping
Right, and it is the second time this list has surfaced. That is a pattern, not a coincidence.
- D

 Both
Record it and resolve it. Neither one substitutes for the other.

THE SIGNAL

The same unverified list has now been flagged by two different prompts in two different modules.

You have run targeted reviews for five defect classes, an adversarial read, and assumption disclosure. Everything comes back clean.

The business owner says the number still looks wrong to her. She cannot say why.

What do you do?

- ☐ A Show her the review results. The query is verified
- ☐ B Run more review passes with different defect classes
- ☐ C Ask her which specific figure looks wrong, and reconcile that one by hand
- ☐ D Rebuild the query from scratch and compare

◆ PAUSE AND DECIDE

What do you do?

A Show the review results

You verified the code does what it says. She is telling you it says the wrong thing.

C Reconcile one figure by hand

Her instinct is evidence from outside the code. One traced figure turns it into a finding.

B More review passes

Review cannot find a missing rule or a wrong definition. More passes will keep coming back clean.

D Rebuild from scratch

Expensive, and it would reproduce the same misunderstanding from the same documents.

THE SIGNAL

She has information the code does not contain. That is the only kind of evidence review cannot reach.

The SQL arc, in one line each

MODULE	THE ONE THING
Context and reconciliation	Documents drift. Diff them against code before you trust either
Spec to SQL	Declare the grain before you write anything
Inherited code	You can recover what it does. Never why
Wrong numbers	Ask for counts, not diagnoses
Performance	Argued equivalent is not proved equivalent
Migration	The dangerous code is the code that needed no translation

WHAT IS LEFT

Two modules left. Everything so far assumed the code was SQL.

Take this to work tomorrow

- ◆ Replace 'review this' with one named defect class. Run it five times.
- ◆ Ask for the input that breaks it, not whether it is correct.
- ◆ Run assumption disclosure at the end of every session that produced code.
- ◆ When someone says the number looks wrong, trace one figure by hand. Believe them first.

THE PAYOFF

The last one is not a prompting technique. It is the most valuable habit in the course.

None of these prompts decided anything.
You did.